

Portfolio — Session Handoff

Project: C:\All code\All project\protfolio2 **Owner:** Aayansh Singh · ayansh0209@gmail.com · Noida, India
Last worked: 22 Aug 2026 **Purpose:** Attach this file at the start of a new chat to pick the build up mid-flight.

First message to send: "Continuing the portfolio build — here's the handoff file. The project is at C:\All code\All project\protfolio2."

Where the build stands



Roughly 72% done. The structure, content and motion are built. What's left is polish, two open design questions, and deployment.

Area	Status
Design system, tokens, type	Done
Header, Hero, Services, About, Contact, Footer	Done
Work carousel (6 projects, pinned, sliding cells)	Done
Two live product demos (graph + sort trace)	Done
Intro sequence, contextual cursor, scroll ruler	Done
3D orbs (6 shapes, morphing)	Done
Real imagery for 4 of 6 projects	Done
CScout + Geospatial screenshots	Missing — you supply
Hero direction (letterform particles?)	Open question
vanlent "PROJECTS" section treatment	Needs clarifying
Mobile polish pass	Not started
Lighthouse / perf pass	Not started
Deploy + custom domain	Not started

Start here — ordered backlog

Work top to bottom; each item is independent.

1. **Delete** `_to_delete/` — 30 seconds, clears dead files and transfer tarballs.

2. **Send CScout and Geospatial screenshots.** Drop them in `public/work/` as `cscout-hero.jpg`, `cscout-t1..t6.jpg`, `cscout-portrait.jpg` (same pattern for `geo-`), then add `image`, `thumbs`, `mobileImage` to those two entries in `lib/content.ts`. Right now they show generated placeholders.
3. **Answer the two open questions** (see §7) so the hero and the PROJECTS section can be finished.
4. **Fix the two placeholder URLs** — the portfolio project's `repo` in `content.ts`, and `metadataBase` in `app/layout.tsx`.
5. **Mobile pass.** The plan file flagged this from the start: wide letter-spacing and grid overlays are desktop-first designs and mobile will fight you. Budget real time. Test the carousel, the intro, and the About stack grid at 390px.
6. **Performance pass.** Three WebGL canvases plus two 2D canvases is a lot. Check on a mid-range laptop, watch the hero's 13,000 points, and confirm the `frameLoop` gating actually stops the work-frame orb when off screen. Target Lighthouse 90+.
7. **Verify accessibility.** Reduced-motion is wired throughout — test it actually holds by toggling the OS setting, not just in code.
8. **Deploy.** Vercel, one click. Then add the link to the GitHub profile README, LinkedIn, and your resume header.

1. What this is

A single-page portfolio, dark and typographic, inspired by **vanlent.dev** (layout system, bordered cell frame, ruler scroll indicator, debug-text decoration) and **bamlab.ch** (dark palette, sparkle particles, monospace nav, pull-quotes). It is *inspired by, not cloned* — no copied artwork, copy, or exact colours.

Run it: `npm run dev` → localhost:3000. Everything is already installed.

Stack

Layer	Choice
Framework	Next.js 16.3.1, App Router, Turbopack
React	19.2.8
Styling	Tailwind CSS v4 (<code>@import "tailwindcss"</code> + <code>@theme</code>)
3D	three ^0.185 + @react-three/fiber ^9.7 + @react-three/drei
Fonts	Jost (display) · JetBrains Mono (mono) · Inter (body), all via <code>next/font/google</code>

No animation library. GSAP, Lenis, Framer Motion are all deliberately absent — the smooth scroll, reveals and pinned carousel are hand-rolled. Don't add one without a reason; it's a talking point in interviews.

2. Architecture

app/

- **layout.tsx** — fonts, metadata, OG tags. `metadataBase` is still a **guessed Vercel URL** — fix before deploying.

- **page.tsx** — composes everything. Order matters: **Intro** and **Cursor** first, then **GridOverlay**, then the fixed hero-object layer (z-index 3), then **main** (z-10), then **ScrollRuler**.
- **globals.css** — all design tokens, type scale, reveal system, grid overlay, checkerboard, scrollbar, reduced-motion block.

lib/

- **content.ts** — **single source of truth for all copy**. Projects, services, stack, links, profile. Edit here, not in components.
- **gfx.ts** — the shared point-cloud kit: GLSL vertex/fragment shaders, sprite generators (soft dot, 4-point star), shape generators (**sphere**, **rippleSphere**, **spikeSphere**, **graphCloud**, **torusKnot**, **lattice**, **globe**, **grid**), **buildCloud()**, **disposeCloud()**, and an unused **loadPortrait()** helper.

components/

Page sections

- **Header.tsx** — fixed bordered nav cells, live IST clock, mobile drawer
- **Hero.tsx** — split headline (MAPPING SYSTEMS / SHIPPING TOOLS), numbered corner labels, viewfinder frame
- **WorkFrame.tsx** — **the big one**. vanlent-style bordered chassis: orb cell / screens / portrait / tech rail / description, with a **< ⌕ name >** control bar. Pinned with **position: sticky**; scroll progress drives all four inner tracks in lockstep
- **Services.tsx** — three tilted cards
- **About.tsx** — CS-student summary + 14-cell stack grid
- **Contact.tsx** — Email / GitHub / LinkedIn
- **Footer.tsx**

Motion & chrome

- **Intro.tsx** — opening sequence. Counter to 100, cycling lines, grid mask-in, clip-path wipe. **SSR-rendered** so it paints on frame one. Once per tab via **sessionStorage**
- **Cursor.tsx** — contextual cursor. Ring + dot; morphs to an arrow over **data-cursor="next"/"prev"**, expands with a label over links
- **SmoothScroll.tsx** — ~40-line inertial scroll driving real **window.scrollTo**, so sticky/anchors/observers all keep working
- **Reveal.tsx** — one IntersectionObserver for every **[data-reveal]**, plus a MutationObserver for late mounts
- **ScrollRuler.tsx** — bottom tick ruler + live **translateY(...)** readout
- **CursorSparkles.tsx** — bamlab-style sparkle trail on a 2D canvas
- **Tilt.tsx** — pointer-driven 3D tilt wrapper with **preserve-3d**
- **SplitText.tsx** — per-character stagger, server component (no JS shipped)

3D & demos

- **ParticleSphere.tsx** — the hero object. Three nested clouds (dust, sparks, counter-rotating inner core) morphing between four forms. Lives on a **fixed** layer so it travels down the viewport as you scroll, then fades
- **ProjectOrb.tsx** — the work-frame object. Ping-pongs between two morph slots so it scales to any number of shapes
- **LiveDemo.tsx** / **LiveGraph.tsx** / **LiveTrace.tsx** — the two working product miniatures (see §5)
- **Shot.tsx** — screenshot slot; real image if present, generated placeholder otherwise

3. Design system

```
--bg:          #0a0a0b    --text:          #ece7e0
--bg-deep:     #070708    --muted:         #8b857d
--surface:     #121113    --faint:         #5a5550
--line:        rgba(232,224,214,0.075)
--line-strong: rgba(232,224,214,0.14)
--accent:      #b08052     (chocolate/bronze)
--accent-soft: #d8b48a
--grid:        84px  (56px under 768px)
--gutter:      clamp(20px, 5vw, 96px)
```

Type: `.display` = Jost, weight 300, uppercase, 0.08em tracking. `.mono` = JetBrains Mono, 0.66rem, 0.19em tracking, uppercase. `.quote` = Jost italic.

Recurring motifs: faint grid overlay with a radial mask · 1px bordered cells · numbered labels (`01.`) · `debug-text` decoration showing real values · checkerboard filler cells · corner brackets.

4. Content model

Everything lives in `lib/content.ts`. A project looks like:

```
{
  index: "01",
  name, field, title, quote,
  blurb,    // paragraph 1 in the description cell – keep ~160 chars
  detail,   // paragraph 2 – keep under ~200 chars or it overflows the cell
  stack: string[],
  rail: string[],           // 3–6 short codes for the vertical tech rail
  facts: [{label, value}], // max 2, they sit at the cell's base
  live?, repo,
  shape: "graph"|"knot"|"lattice"|"sphere"|"globe"|"grid",
  demo?: "graph"|"trace", // only the two flagship projects
  image?, thumbs?: string[], mobileImage?,
  tint: [accent, deep],
}
```

Six projects, in order: CodeMap AI · Interactive Algorithm Tutor · This Portfolio · CScout · E-Commerce Platform · Geospatial Risk Mapping (team project, last).

Adding a project: append to the array, drop `<prefix>-hero.jpg`, `<prefix>-t1..t8.jpg`, `<prefix>-portrait.jpg` into `public/work/`, use the `thumbs("<prefix>")` helper. The carousel maths derives everything from `projects.length` — nothing else to change.

Verified numbers — do not inflate. CodeMap AI has 52 GitHub stars and did 744 visitors / 1,127 page views in month one. The earlier plan explicitly warned against writing "1000+ users".

5. The two live demos

These are the differentiator. Neither is a video or a gif — both compute in real time.

- **LiveGraph.tsx** (CodeMap AI) — a 3D dependency graph. `auth.ts` hub, IMPORTED BY on one side, IMPORTS on the other, right-angle connectors with pulses travelling them. Canvas 2D with a hand-rolled perspective projection. Labels only render for front-facing nodes so rotation declutters instead of piling up.
 - **LiveTrace.tsx** (Interactive Algorithm Tutor) — a real bubble sort on `[5,2,9,1,6]`. The 17 steps come from actually running the algorithm and recording every compare and swap. Code panel with a live line highlight, extruded 3D bars that lean with the pointer, step counter, progress bar.
-

6. Gotchas — these cost real time, don't rediscover them

1. **overflow: hidden** does not clip 3D-transformed children. The carousel cells use `clipPath: "inset(0)"` instead. If you add a cell with `Tilt` inside, it needs the same.
 2. **overflow-x: hidden** on **body** breaks **position: sticky** for descendants — it makes body a scroll container. It's set to `overflow-x: clip`, which doesn't. Don't change it back.
 3. **React Compiler lint** (`react-hooks/immutability`, `react-hooks/refs`) fights imperative WebGL. `ParticleSphere.tsx` and `ProjectOrb.tsx` carry a file-level `eslint-disable react-hooks/immutability` with a comment explaining why. Never assign `ref.current` during render — do it in an effect.
 4. **setState directly in an effect body** is a lint error in this config. Wrap it in `requestAnimationFrame`, or use `useSyncExternalStore` for client-only reads (that's what the media-query checks do).
 5. **next/font** needs network at build time. Offline builds fail on Google Fonts.
 6. **Headless Chromium reports prefers-reduced-motion: reduce**. Screenshot tooling must pass `reducedMotion: 'no-preference'` or all the motion silently disables. Playwright's `animations: 'disabled'` also fast-forwards transitions, which hides the intro.
 7. **_to_delete/** at the project root holds dead files and transfer tarballs. It's excluded from tsconfig and eslint. **Just delete the folder** — the desktop bridge can't delete, only move.
-

7. Open items

Asked for, not built:

- **"Add that type of project from the last picture"** — the reference screenshot was vanlent's PROJECTS section with `clip-path: inset(...)` debug text and corner brackets. Never clarified whether that meant an animated section-intro before the work carousel, or something else. **Ask first.**
- **A richer hero.** The current object is three nested clouds morphing between four shapes. The bigger idea, not yet built: make the particles fill the *letterforms of the name*, the way bamlab fills "BAM!". That's a much larger visual step than making the sphere denser.

Housekeeping:

- Portfolio project's `repo` in `content.ts` points at the GitHub profile — swap for the real repo once pushed.

- `metadataBase` in `layout.tsx` is a guess.
 - Geospatial Risk Mapping copy was written from file names (`lstm_pipelining.py` , `run_gan_pipeline.py` , `run_smote_pipeline.py` , `predict_map.py`) because the repo has no README description. Worth checking with the team.
 - E-commerce screenshots were pulled from the repo's own README assets (`frontend/public/assets/image-1..7.png`).
 - Delete `_to_delete/` .
-

8. Making a 3D model of yourself

Gemini cannot do this — it outputs no meshes. Neither can ChatGPT or Claude.

Scanning (best likeness): photogrammetry — 60–150 photos in a full circle at three heights, even light, plain background, hold still. Process in RealityCapture (free non-commercial) or Meshroom. Phone apps: Polycam, KIRI Engine, Scaniverse. iPhone Pro LiDAR takes ~2 minutes and is good enough at portfolio scale.

AI image → 3D: Meshy, Tripo, Hunyuan3D (open weights, strongest free), Rodin, Stable Fast 3D. They reconstruct a convincing body from one photo but the *face* rarely resembles you specifically.

Rigged avatar: Ready Player Me — selfie → rigged GLB in a minute, drops into three.js with an idle animation.

The pipeline that fits this site. A photoreal textured mesh would clash with a page made of dust and thin bronze lines. Render the mesh **as points** instead:

1. Get a GLB by any method above
2. Blender: decimate to ≤50k tris, apply transforms, centre at origin, scale to ~2 units tall, export GLB
3. Run it through three.js `MeshSurfaceSampler` to scatter ~30k points over the surface, brightness from the texture
4. Write `[x, y, z, brightness]` as a Float32 binary

That drops straight into `lib/gfx.ts` as another morph target — no new rendering code, ~500KB instead of a 20MB textured GLB. A previous attempt did exactly this from a passport photo (segmented background, contrast-weighted sampling, head-bulge depth) and it read well, but it was removed on request because the face didn't look good in context.

9. Unrelated but live: CodeMap AI infrastructure

The production worker went down mid-session with `Bad credentials` from the GitHub API — an expired personal access token, fixed by regenerating and restarting pm2 with `--update-env` . Two things to remember:

- `pm2 restart` reuses the environment captured when the process **first started**. Editing `.env` does nothing without `--update-env` , and `pm2 save` afterwards so a reboot doesn't restore the old env.
- The worker should assert its token at boot rather than failing 5% into every job, and should not surface GitHub's raw "Bad credentials" to visitors — it reads like *their* credentials are wrong.

Security: a screenshot of the full `.env` was shared in that session. Rotate the Backblaze B2 keys (`R2_ACCESS_KEY_ID` , `R2_SECRET_ACCESS_KEY`), the `UPSTASH_REDIS_REST_TOKEN` , and the GitHub token if not already done. B2 first — object storage credentials are the most costly to leak.